1. 1000 1.8(8)

2. 2^{100} $\log_{\frac{3}{2}} n$ $n^{\frac{3}{2}}$ $(\frac{3}{2})^n$ 1.10

3. $\frac{n}{2}$

4. 3.6

1）必要性。按照题意，当i<j<k时，进栈顺序是i,j,k,这3个元素出栈的相对顺序是pi,pj,pk.例如，当i=1,j=2,k=3时，一个合理的出栈序列是pi=2,pj=3,pk=1.如果如果pj<pk<pi成立，意味着出栈顺序为3，1，2，这恰恰是不可能的。当较大的数首先出栈时，那些较小的数都是降序压在栈内的，如2，1，这些数不可能如1，2那样正序出栈

2）充分性。如果pj<pk<pi成立，表明当i<j<k时各元素进栈，出栈后的相对顺序为pj，pk，pi。

下面做具体分析：

当i<j时，pj<pi，表明pj在pi进栈后进栈并压在pi上面，并且pj在pi出栈前出栈。

当j<k时，pj<pk,表明pj必须在pk进栈之前就出栈，否则pj就被压在pk下面了。

当i<k时，pk<pi,表明pi时先于pk进栈的

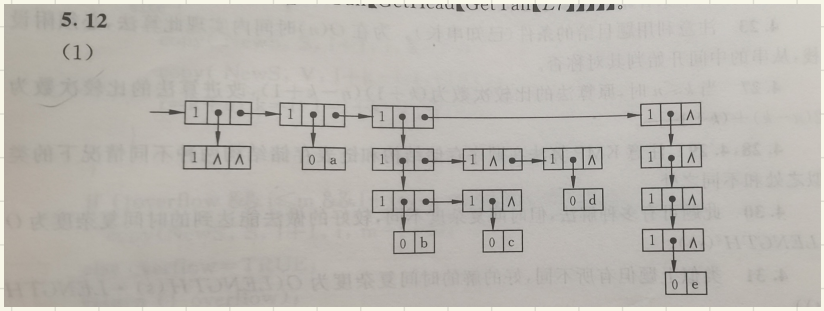
综上所述可知，这与正确的出栈顺序pi<pj<pk相矛盾。

5. 4.7

j	1	2	3	4
串s	a	a	a	b
next[j]	0	1	2	3

6. 5.10(8) (d)

7. 5.12 (1)



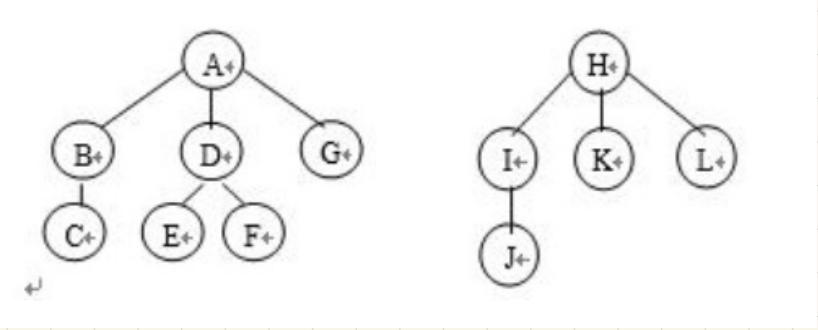
8. 6.8

$n = n_0 + n_1$ ① → 节点总数

$n = kn_1 + 1$ ② → 度

由①② => $n_0 = (k-1)n_1 + 1$

9. 6.24



10

```
#include <iostream>
typedef struct BiTNode{
    char data;
    struct BiTNode *lchild,*rchild;
}BiTNode,*BiTree;
using namespace std;
void CreatBiTree(BiTree &T){
    char ch;
    cin>>ch;
    if(ch=='#'){
        T=NULL;
    }else if(ch!='#' && ch!='!'){
        T=new BiTNode;
        T->data=ch;
        CreatBiTree(T->lchild);
        CreatBiTree(T->rchild);
    }
}
void Exchange(BiTree &T){
    if(T){
        if(T->lchild==NULL && T->rchild==NULL){
            return;
        }else{
            BiTree temp;
            temp=T->lchild;
            T->lchild=T->rchild;
            T->rchild=temp;
        }
        Exchange(T->lchild);
        Exchange(T->rchild);
    }
}
int main(){
    BiTree T;
    CreatBiTree(T);
    Exchange(T);
}
```